TryHackMe // Write-Up

Conti Ransomware

Exchange Server Compromise · Splunk SIEM Investigation · Ransomware Forensics

CVE-2020-0796 · CVE-2018-13374 · CVE-2018-13379

Author: Mojalefa “Jeff” Letsoara // Date: 25 - 26 March 2026 // Difficulty: Medium

Challenge/Room Overview

FIELD	DETAIL
Room Name	Conti
Platform	TryHackMe
Category	Security Operations / Incident Response / Splunk
Scenario	Exchange Server compromised with Conti ransomware. Employees unable to access Outlook. Use Splunk to investigate.
CVEs	CVE-2020-0796 (SMBGhost) · CVE-2018-13374 · CVE-2018-13379
Primary Tool	Splunk 8.2.2 (via TryHackMe AttackBox)
Log Sources	WinEventLog:Security, Sysmon/Operational, IIS logs, WinEventLog:System, WinEventLog:Application
Threat Intel	SentinelOne Anthology (Conti) + Securin.io CVE analysis - not the room walkthrough
Completed	25 - 26 March 2026 (split over two AttackBox sessions due to KV Store crash)

Environment - What Happened Before Investigation Started

This room did not start cleanly. The first AttackBox session launched Splunk (SPLUNK_1) but almost immediately the KV Store process crashed - three consecutive errors appeared in the Splunk messages panel (SPLUNK_2). Within minutes, Splunk disconnected from the server entirely (SPLUNK_3). Rather than fighting a broken environment, I terminated the session and started a fresh AttackBox.

This is a real-world lesson: in an actual SOC, you do not begin an investigation on a degraded SIEM. You confirm your tooling is healthy first I believe. The second session came up clean and stable on a new IP.

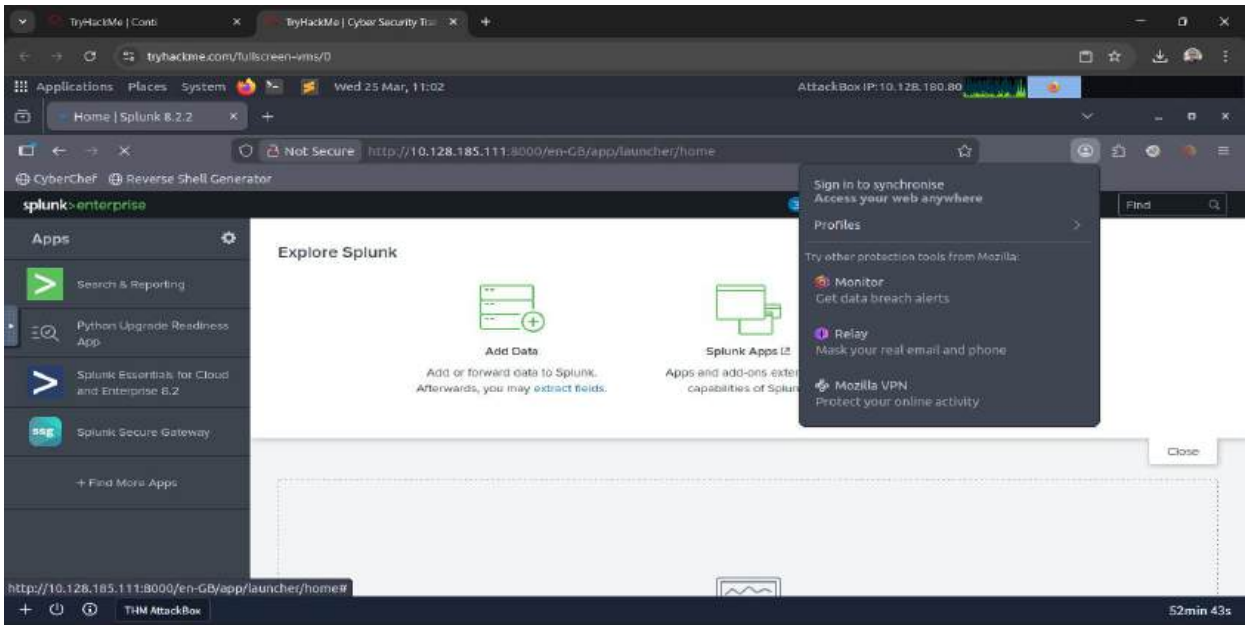

Figure 1: Splunk home - first AttackBox session launched successfully

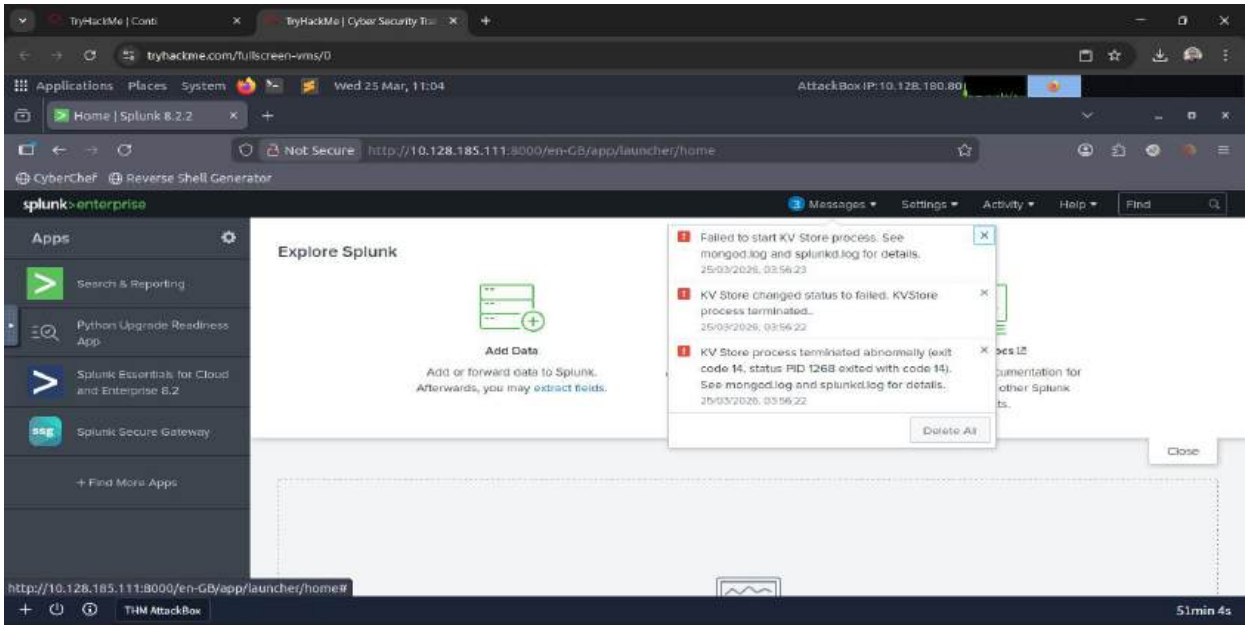

Figure 2: KV Store process crashed - three errors appeared immediately after launch

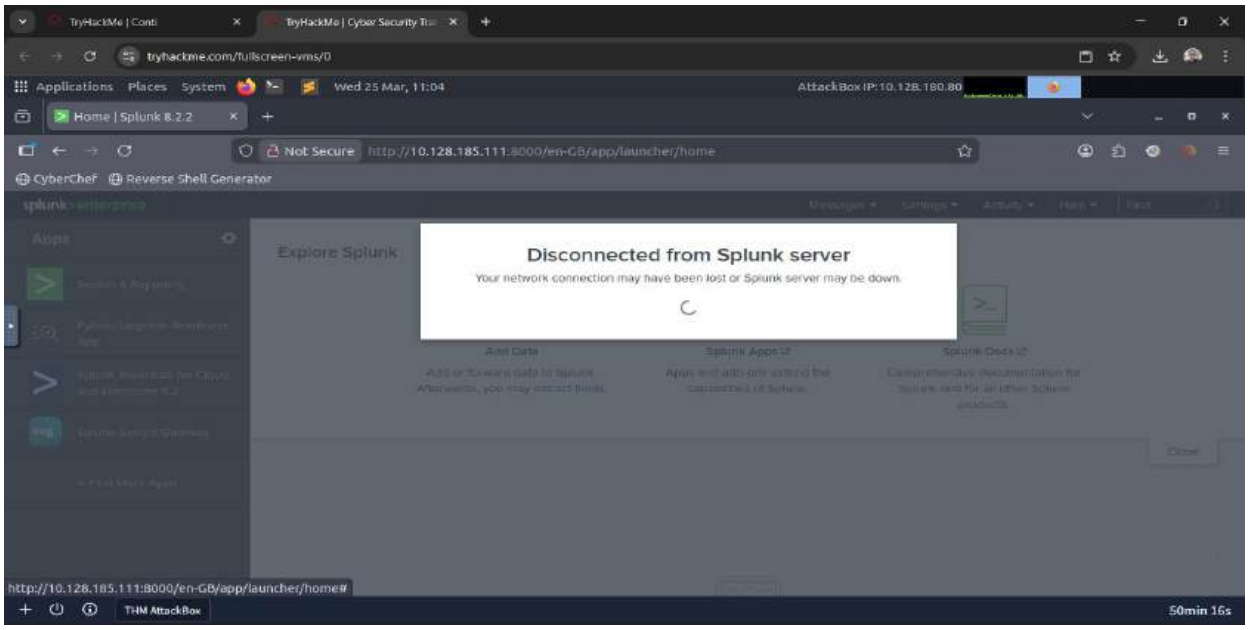

Figure 3: Splunk server disconnected entirely - session terminated and restarted

Scenario - The Case

Employees reported they could not log into Outlook. The Exchange system administrator also lost access to the Exchange Admin Center. First responders found unusual readme files scattered across the Exchange server, the ransomware note. The investigation task: use Splunk to find out how the attacker got in, what they did and what they left behind.

The attacker compromised an Exchange server, deployed Conti ransomware, and dropped readme.txt ransom notes across the filesystem. CVEs exploited: CVE-2020-0796, CVE-2018-13374, CVE-2018-13379.

The Investigation - Step by Step

▸ Step 1 - Splunk versus the Data

After the fresh AttackBox started, I navigated to Search & Reporting and ran the broadest possible search to understand what data was available.

```
index=*
```

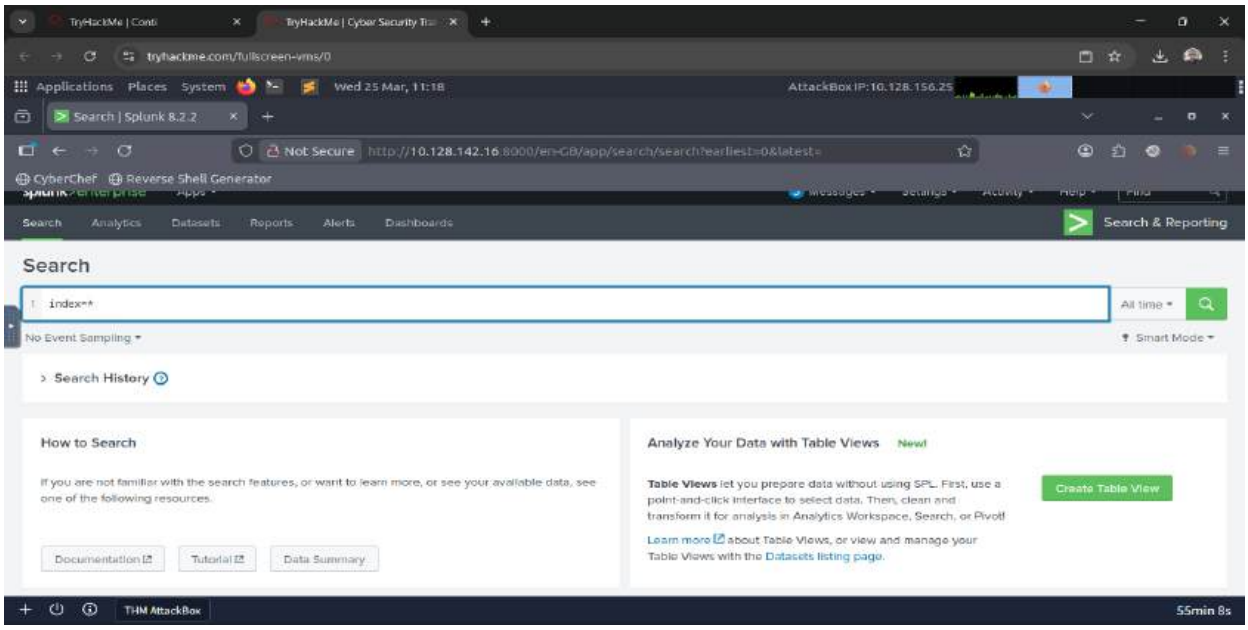

Figure 4: Search & Reporting - fresh session, index= ready to run*

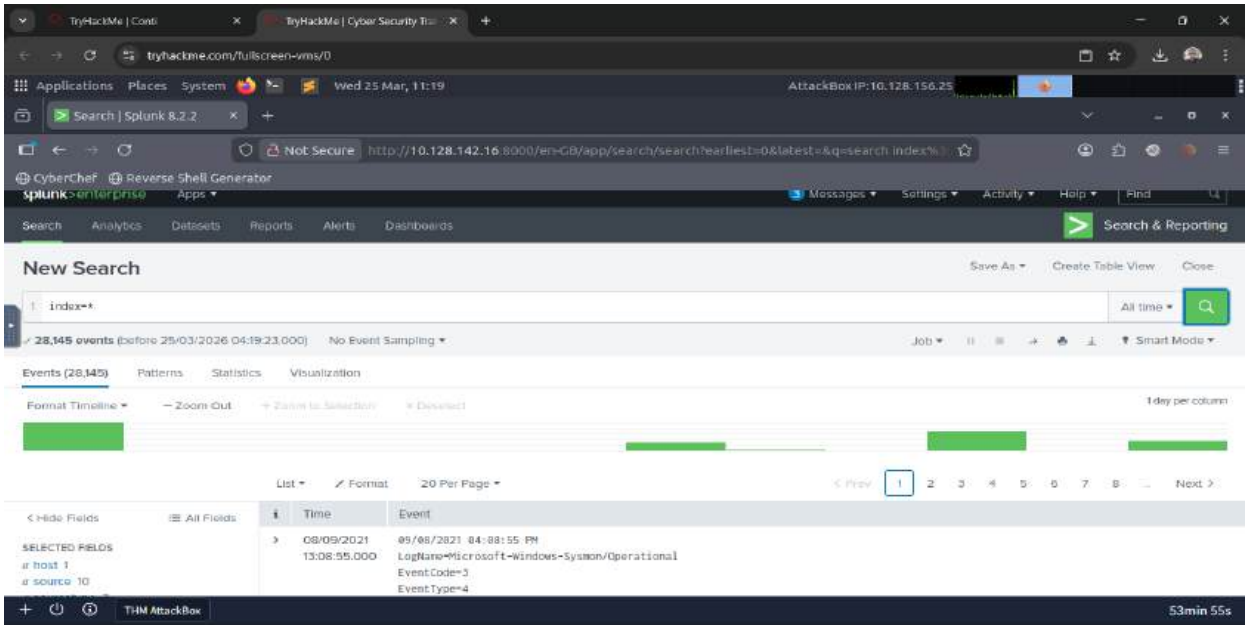

Figure 5: index= returned 28,145 events across all time*

28,145 total events. Before hunting for anything specific, I explored the field values to understand what log sources were present. This is standard practice - you do not query blindly, you first learn the shape of the data.

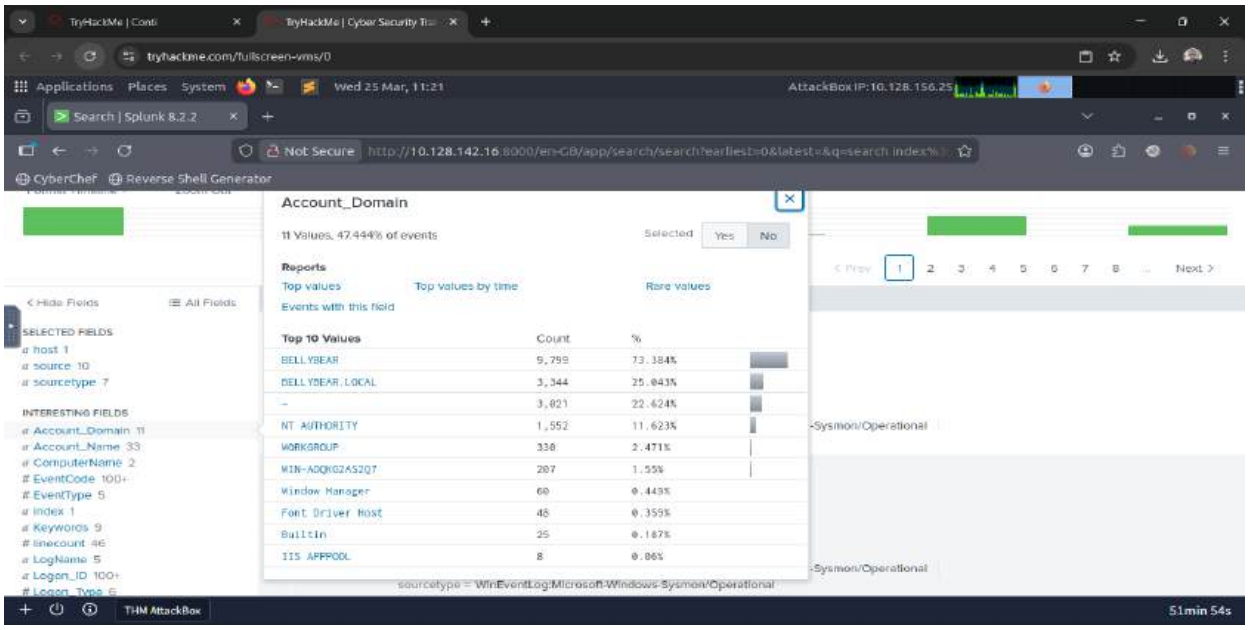

Figure 6: Account_Domain field — BELLYBEAR domain dominant at 73%, confirming the environment context

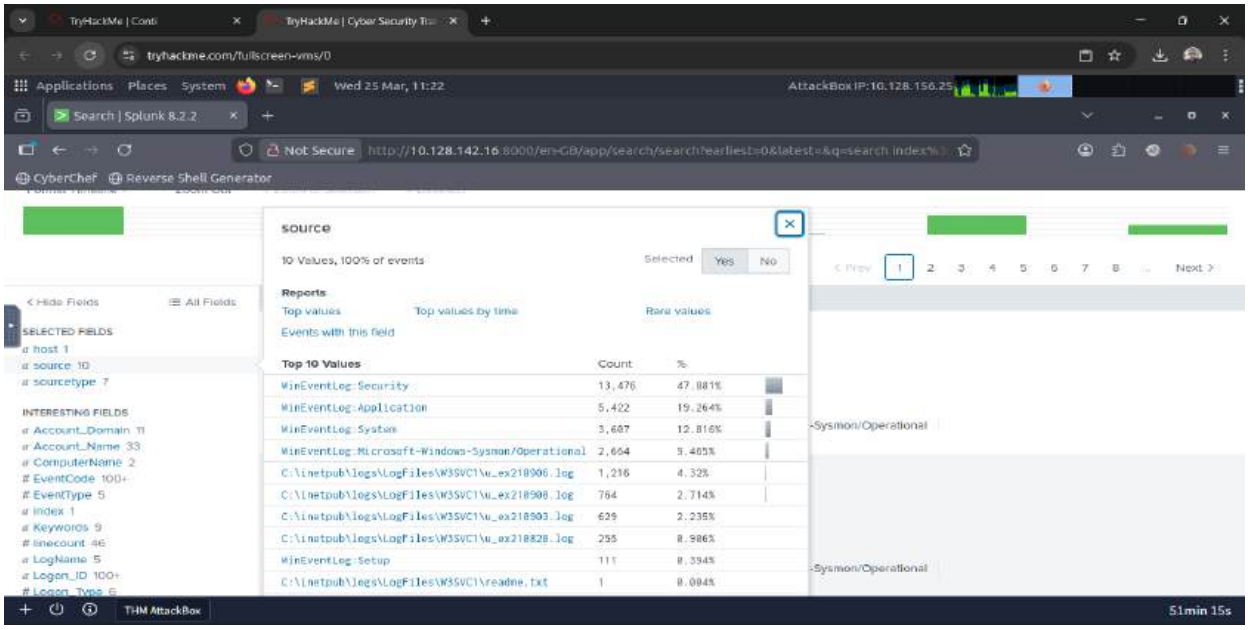

Figure 7: Source field - 10 log sources. IIS logs confirmed that Exchange is web-facing. Sysmon operational logs present.

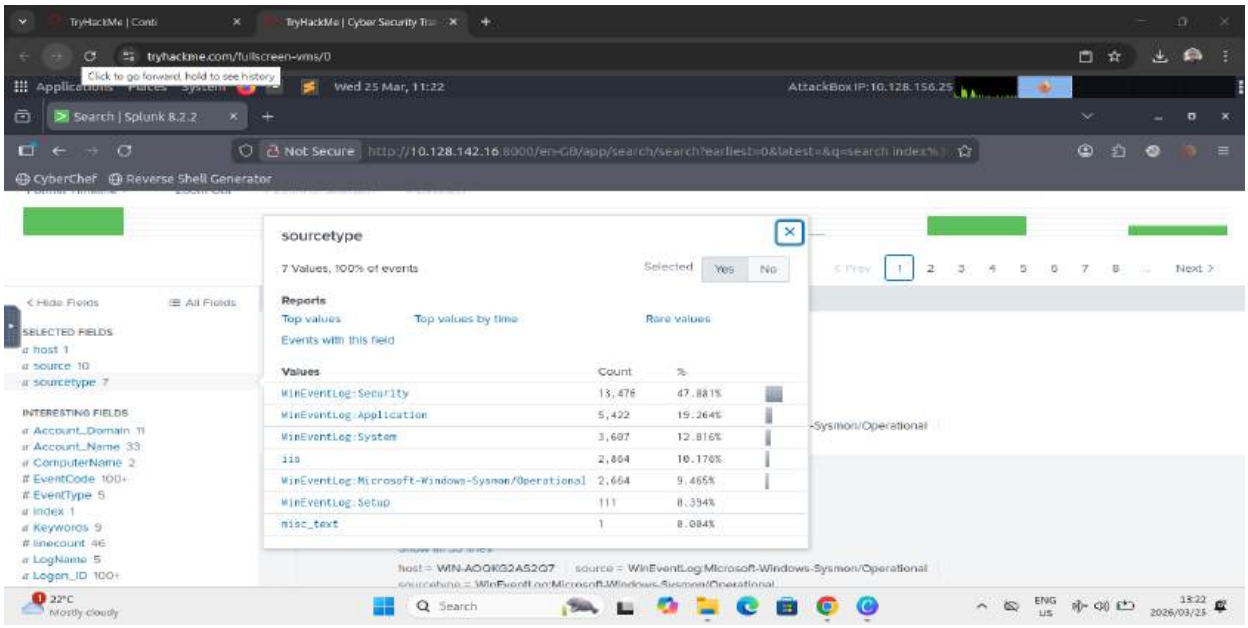

Figure 8: Sourcetype breakdown - Security logs 47%, Application 19%, System 12%, IIS 10%, Sysmon 9%

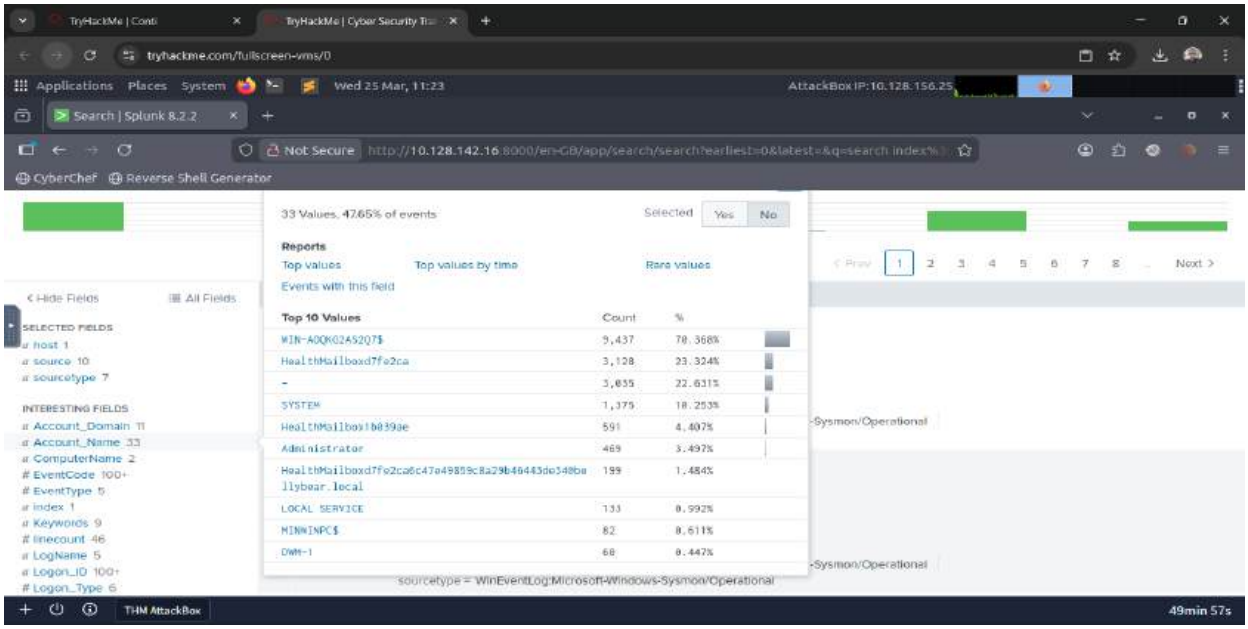

Figure 9: Account_Name values - WIN-AOQKG2AS2Q7 machine account dominant, Administrator account visible

> *SOC Note: The presence of IIS logs alongside Sysmon is significant. IIS = Exchange web interface logs (OWA, EAC). Sysmon = endpoint process and network telemetry. Together they give you both the web attack surface and what happened on the host after initial access.*

▸ Step 2 - Where is the Ransomware? (Finding cmd.exe in the Wrong Place)

The first question asked for the ransomware location, described as a common Windows binary in an unusual location. I enabled the Image field (the process image path that Sysmon logs) and ran a top values analysis across all 28,145 events.

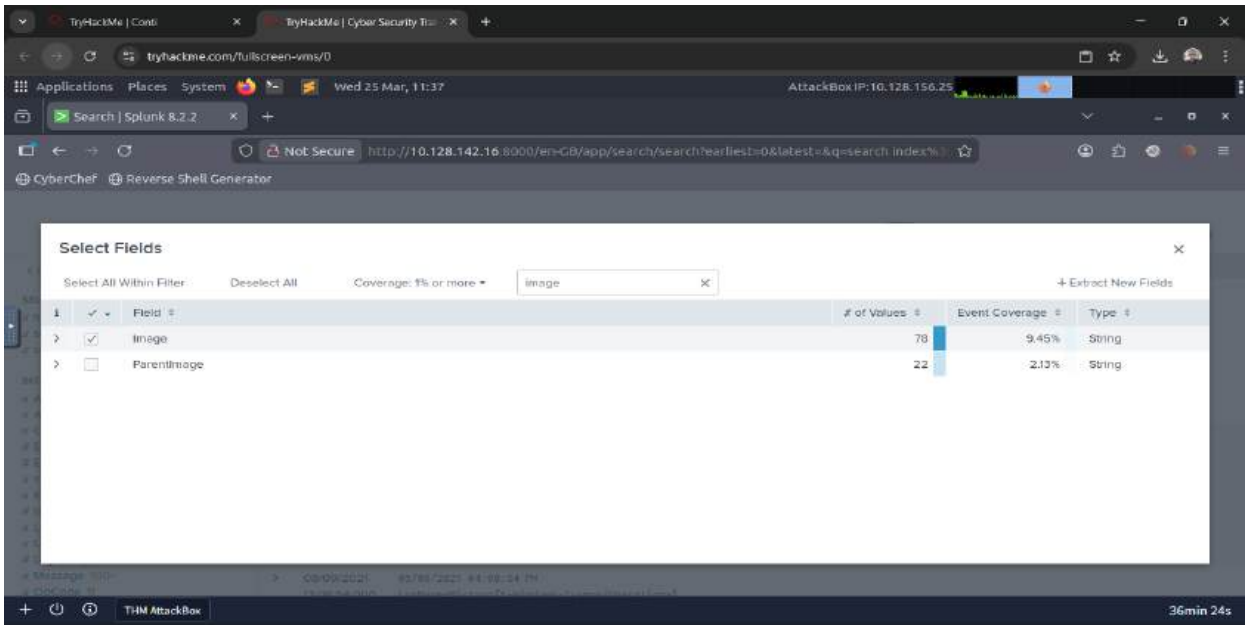

Figure 10: Enabling the Image field - 78 unique values, 9.45% event coverage from Sysmon

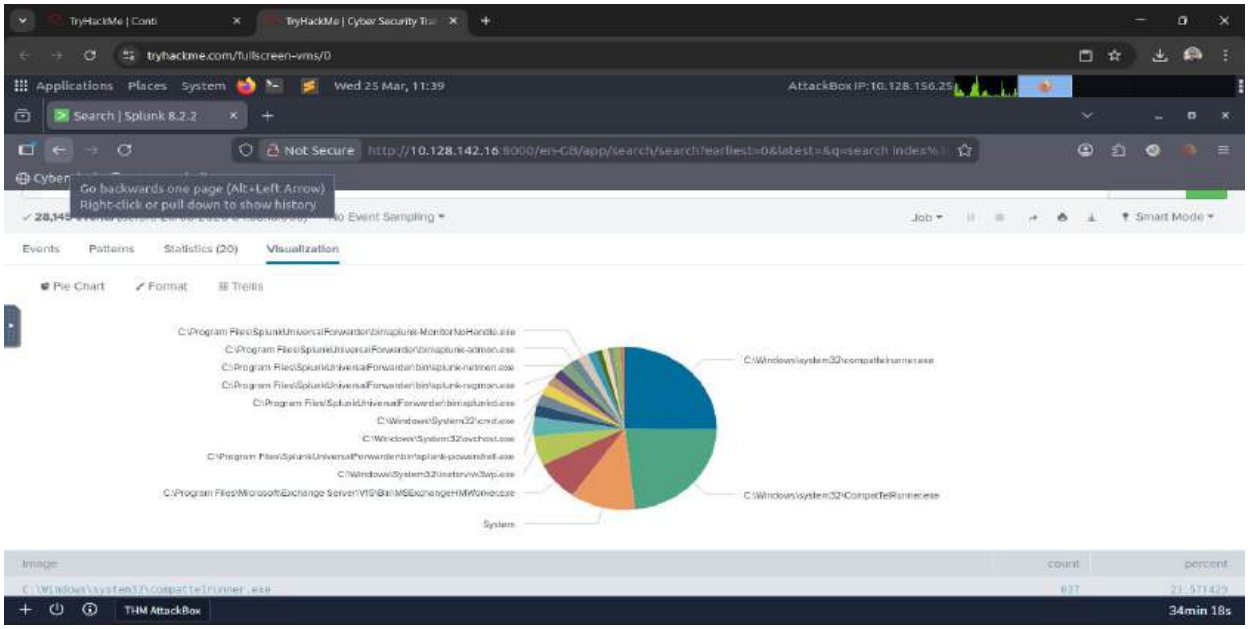

Figure 11: Pie chart of top Image values - dominated by legitimate Windows processes

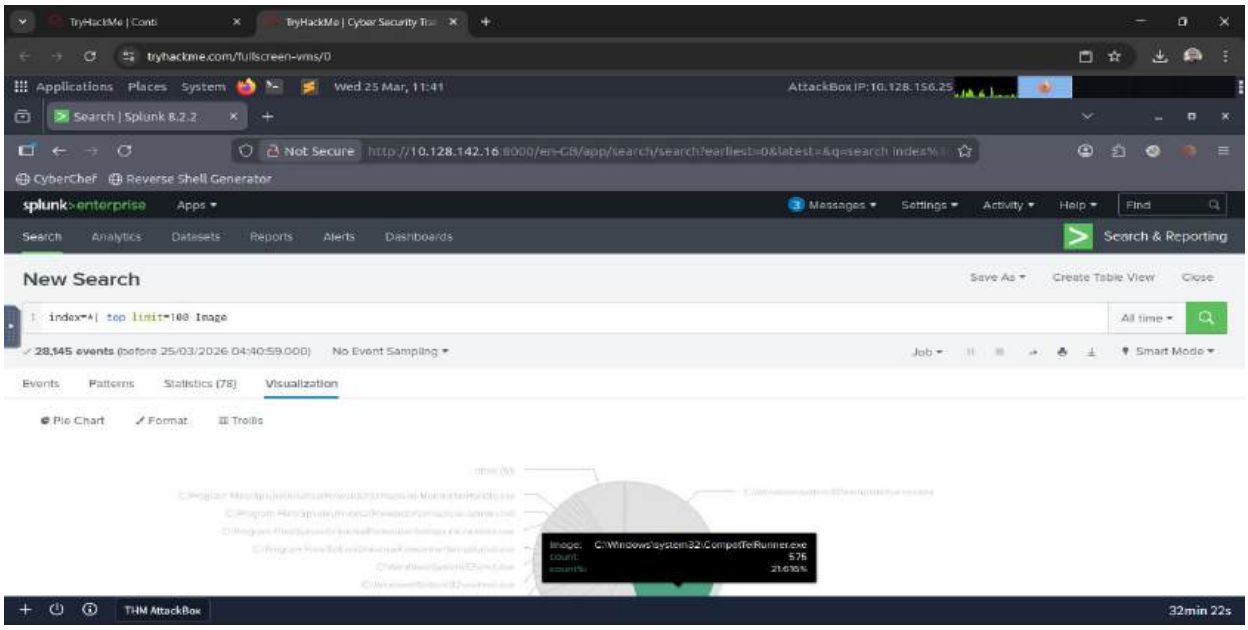

Figure 12: Refined search: index= | top limit=100 Image - expanding to see all process paths*

Scanning down the statistics table, most entries were expected - System32 binaries, Exchange server processes, Splunk forwarder processes. Then one entry broke the pattern completely.

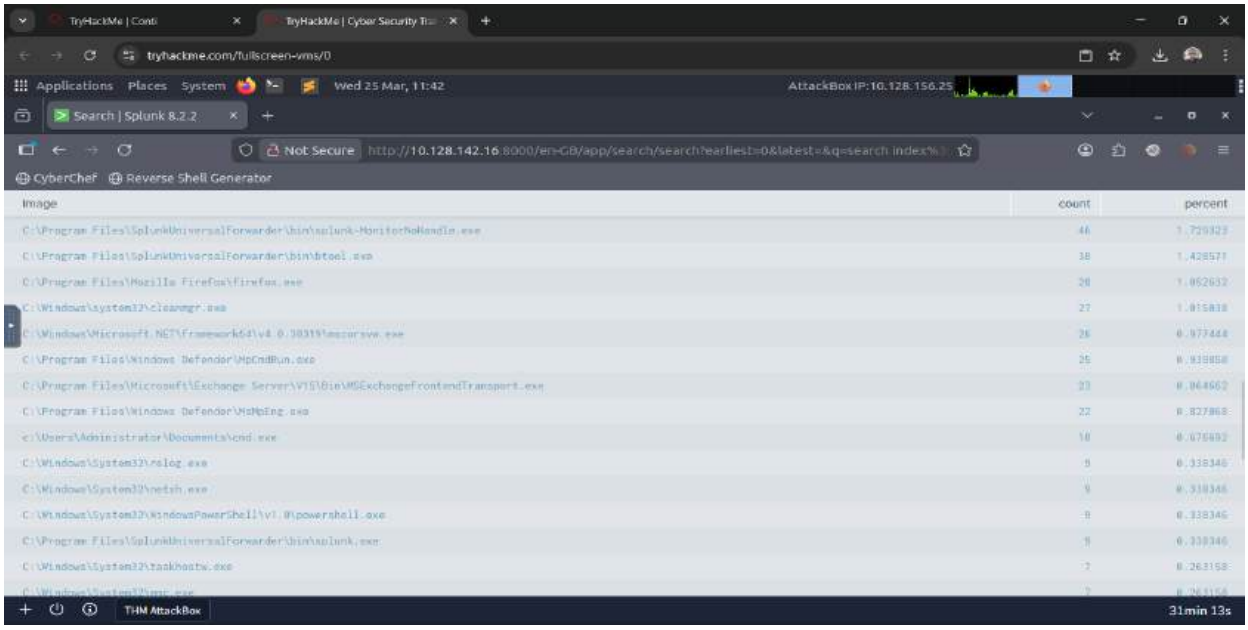

Figure 13: Statistics table — c:\Users\Administrator\Documents\cmd.exe appears 18 times. cmd.exe does not belong in Documents.

cmd.exe is a legitimate Windows binary. Its home is C:\Windows\System32\cmd.exe. Finding it copied to C:\Users\Administrator\Documents\ means the attacker placed a version of cmd.exe - or something posing as cmd.exe in a user-writable directory. This is the ransomware executable.

Ransomware can be located in `\Users\Administrator\Documents\cmd.exe`

SOC Note: Masquerading as cmd.exe is a classic LOLBin technique. A renamed or trojanised cmd.exe in a user directory bypasses application control policies that only monitor System32. Always compare process image paths against expected locations.

▸ Step 3 - Sysmon Event ID for File Creation

Rather than guessing, I looked it up directly.

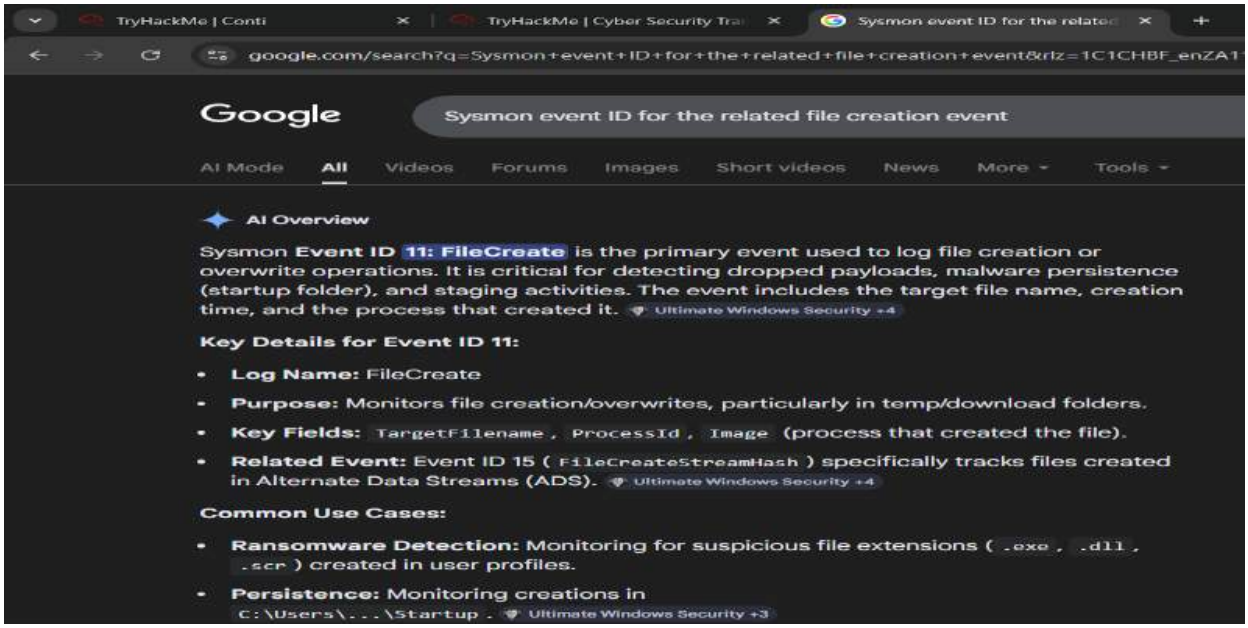

Figure 14: Google search - Sysmon Event ID 11 is FileCreate, used to detect dropped payloads and staging activity

Sysmon Event ID 11 (FileCreate) fires every time a file is created or overwritten. It captures the TargetFilename, the ProcessId, and the Image (process that created the file). This is exactly the event type that would log the ransomware dropping readme.txt files across the filesystem.

Sysmon Event ID for file creation is `11`

▸ Step 4 - MD5 Hash of the Ransomware

With the ransomware path confirmed, I filtered directly to events associated with that Image path and searched for hash values.

```
index=* Image="c:\\Users\\Administrator\\Documents\\cmd.exe"
# Returns 22 events

index=* Image="c:\\Users\\Administrator\\Documents\\cmd.exe" md5
# Filters to the event containing hash data
```

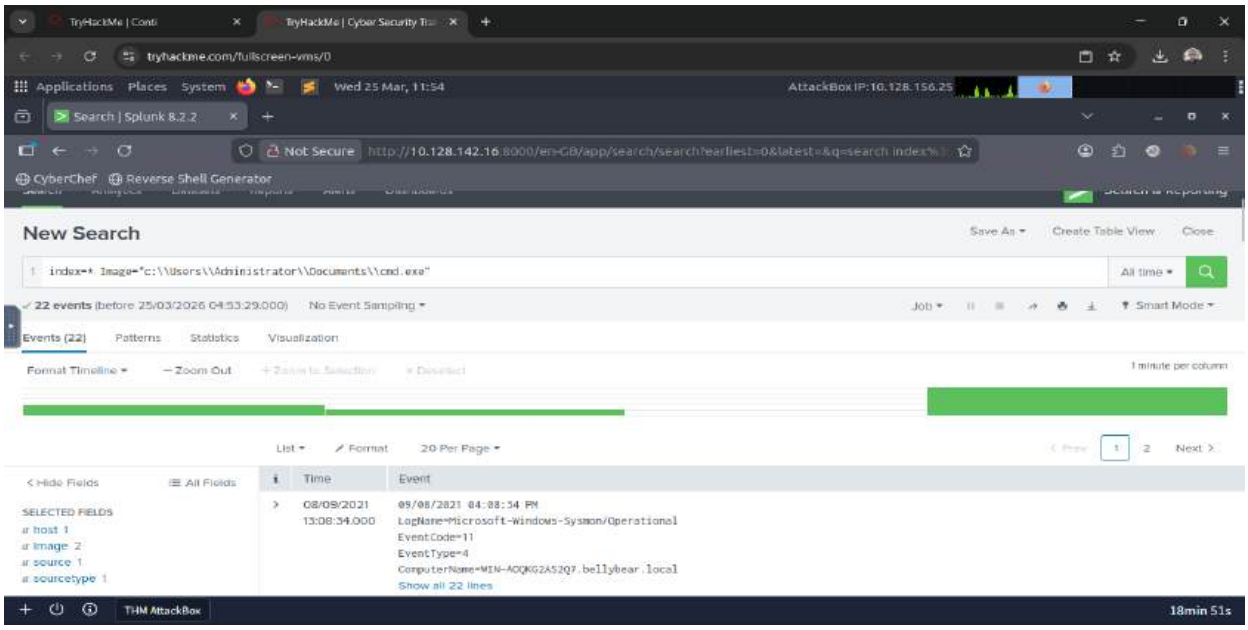

Figure 15: 22 events associated with the malicious cmd.exe Image - EventCode=11 visible, confirming file creation activity

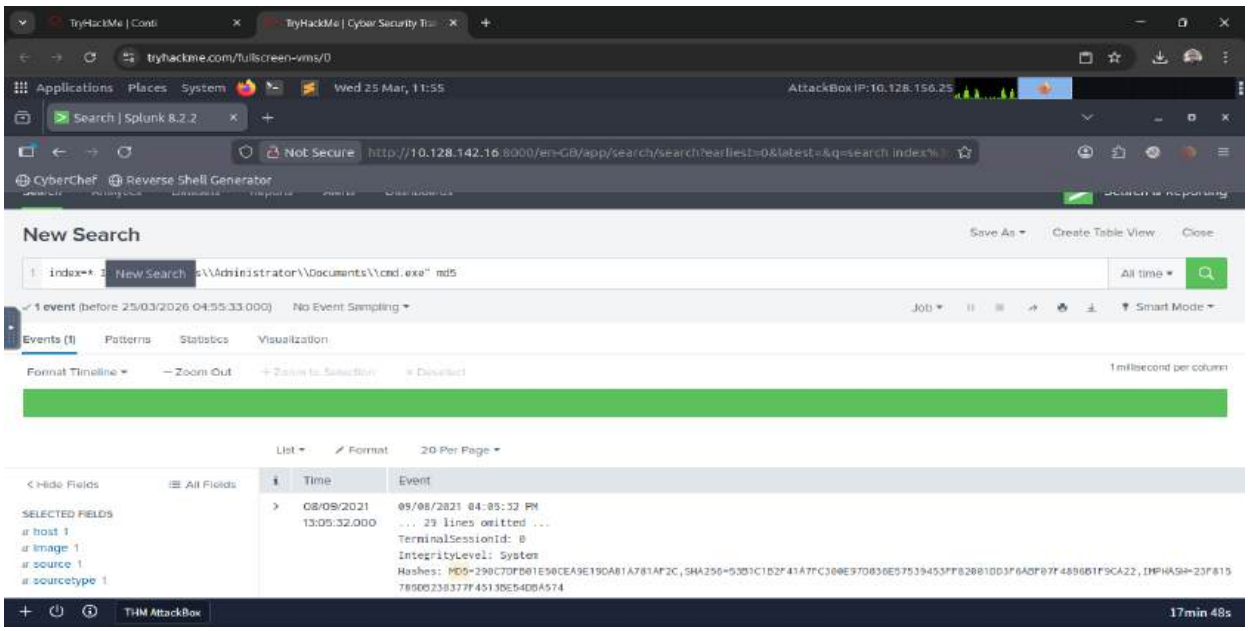

Figure 16: Adding 'md5' to the search -1 event returned. Hash clearly visible in the Hashes field.

MD5 hash `290C7DFB01E50CEA9E19DA81A781AF2C`

SOC Note: Submit this hash to VirusTotal immediately in a real investigation. MD5 alone is weak but it gives you a fast first-pass check. The SHA256 in the same event is more reliable for definitive identification.

▸ Step 5 - What File Was Saved to Multiple Locations?

Ransomware always drops a ransom note. I filtered to EventCode=11 (all file creation events) and examined the TargetFilename field to find files being written across multiple directories.

```
index=* EventCode=11
```

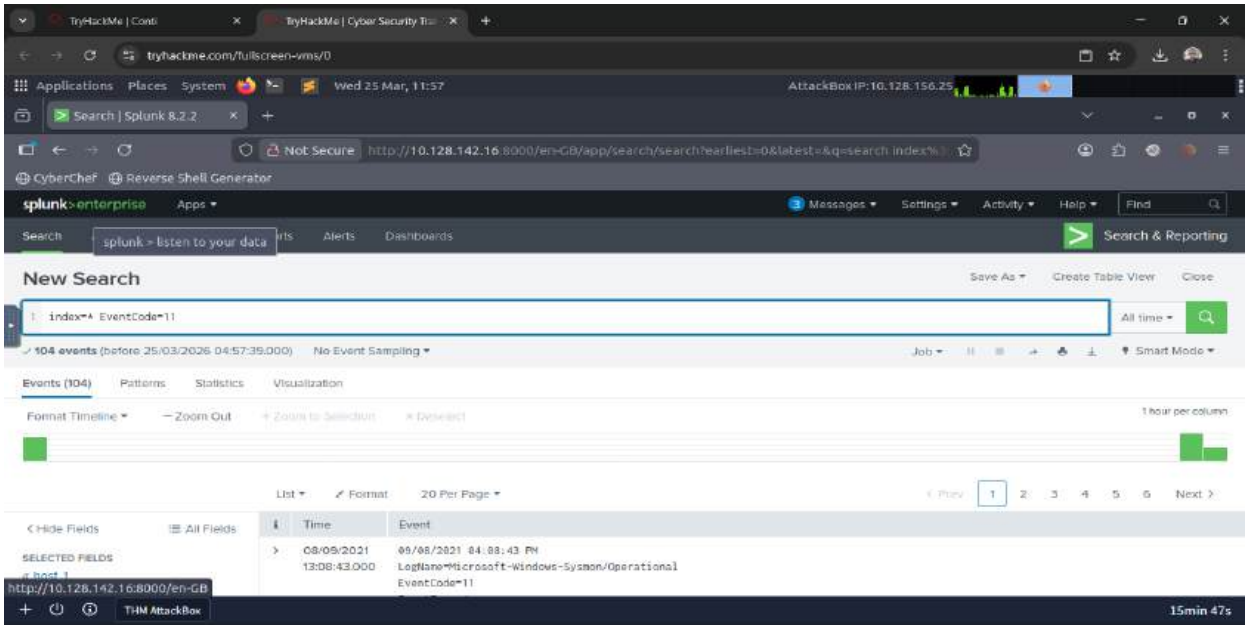

Figure 17: 104 file creation events under EventCode=11 - filtering scope now focused on file drops

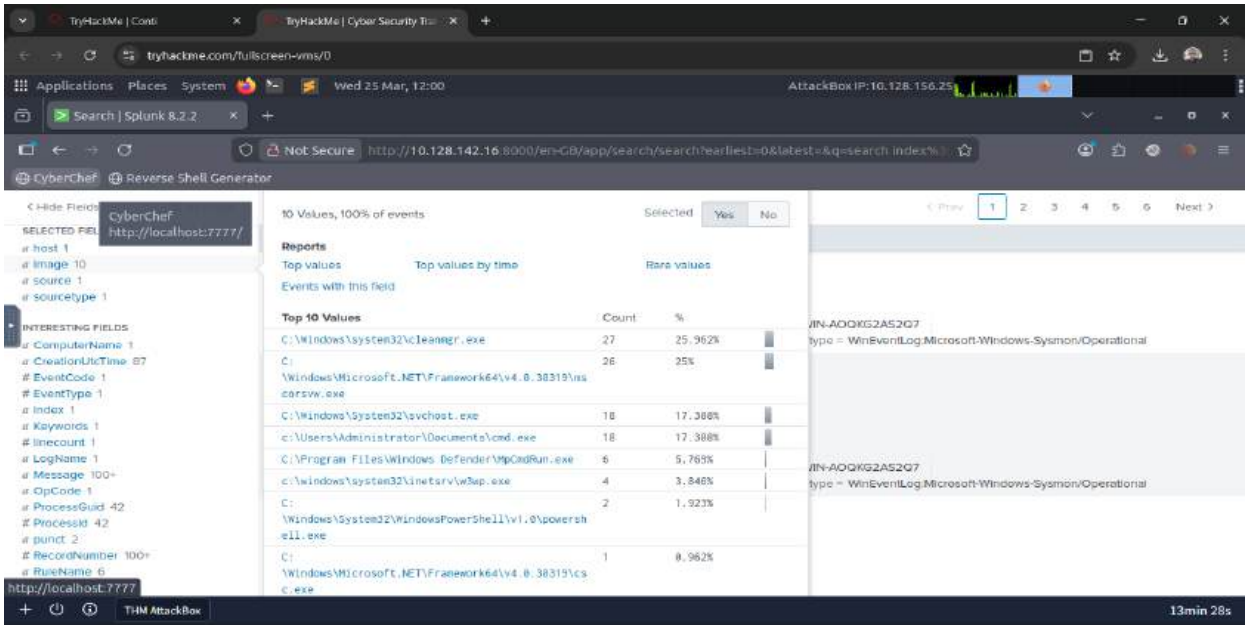

Figure 18: Image field within EventCode=11 results - cmd.exe and svchost.exe prominent alongside legitimate processes

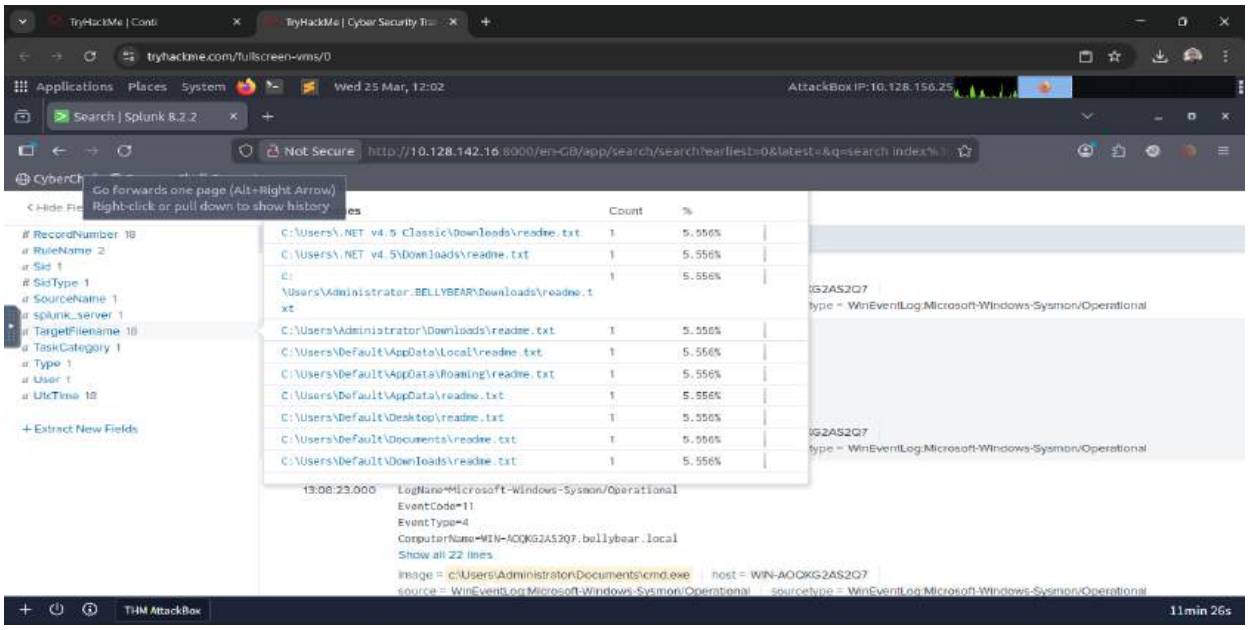

Figure 19: TargetFilename field - readme.txt appearing across Downloads, Desktop, Documents, AppData\Local, AppData\Roaming, AppData (root), and more

readme.txt was being dropped into every user directory systematically. This is exactly how Conti operates, it ensures the ransom note is visible everywhere the victim might look, maximising psychological pressure.

File saved to multiple locations `readme.txt`

▸ Step 6 - The New User the Attacker Added

Attackers frequently create backdoor accounts after gaining access. This gives them a persistent login, even if the initial exploit is patched. I searched for user creation activity.

```
index=* user /add
```

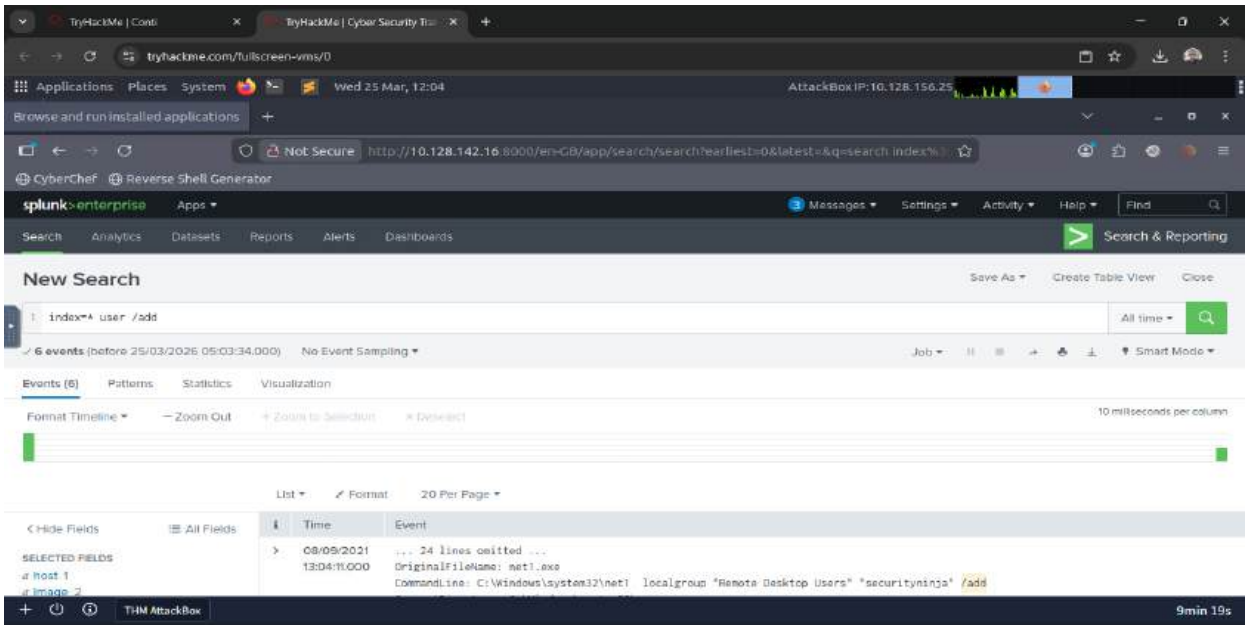

Figure 20: index= user /add - 6 events. CommandLine shows securityninja being added to Remote Desktop Users group.*

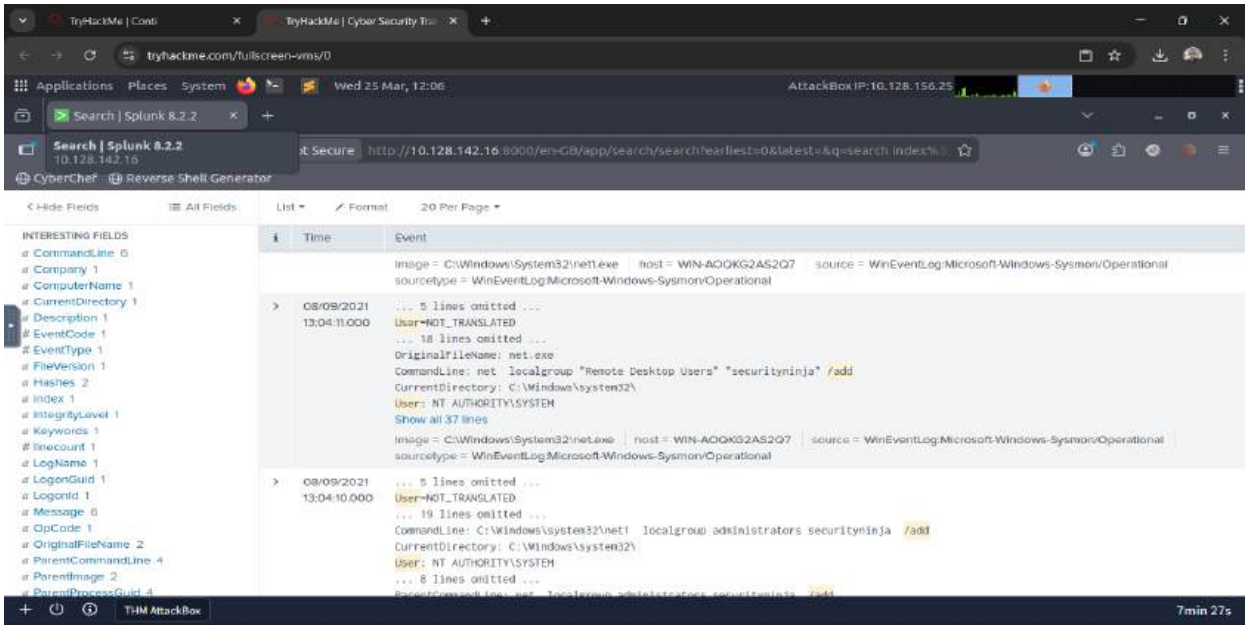

Figure 21: Expanded events - two commands visible: net localgroup 'Remote Desktop Users' securityninja /add AND net localgroup administrators securityninja /add

The attacker didn't just create the account, they added it to both the Remote Desktop Users group (enabling RDP access) and the local Administrators group (giving it full privileges). This is a full persistence and privilege setup in two commands.

To confirm the exact first command and timestamp I used Event ID 4720 which is the Windows Security log event for user account creation.

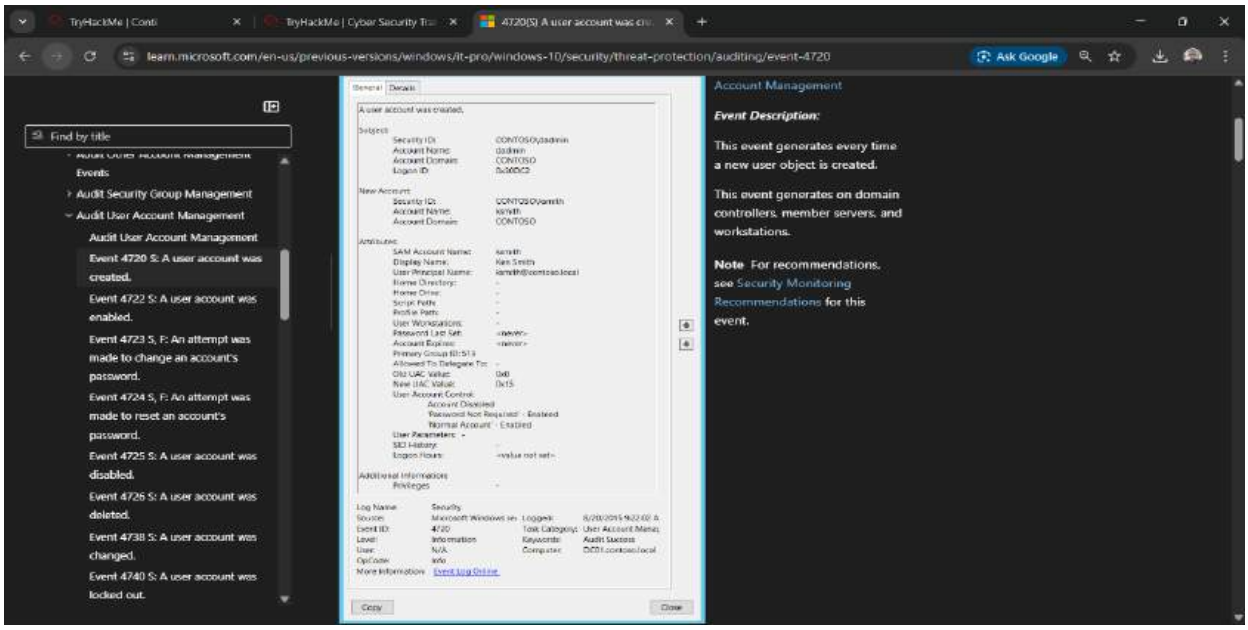

Figure 22: Microsoft docs - Event 4720 fires every time a new user account is created

```
index=* EventCode=4720
```

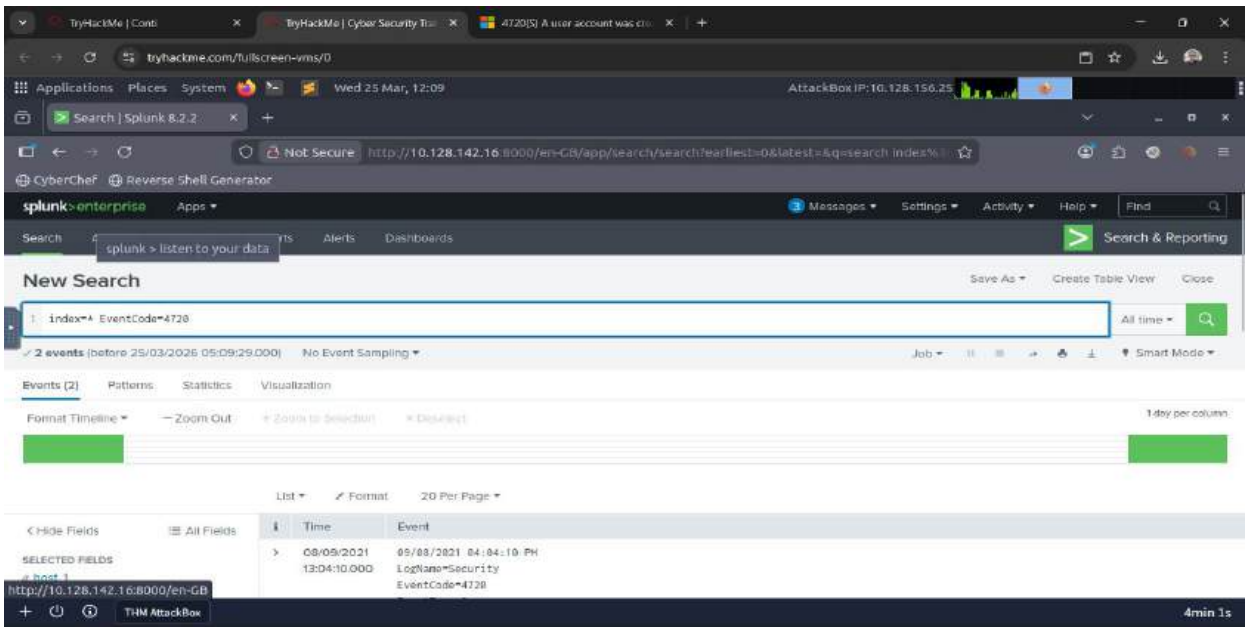

Figure 23: index= EventCode=4720 - 2 events returned. Two timestamps: 09/08/2021 13:04:10 and 28/08/2021 14:48:46*

Two account creation events. I needed to narrow to the correct one - the first creation, not a later modification. I used the Date & Time Range filter to isolate around the first timestamp.

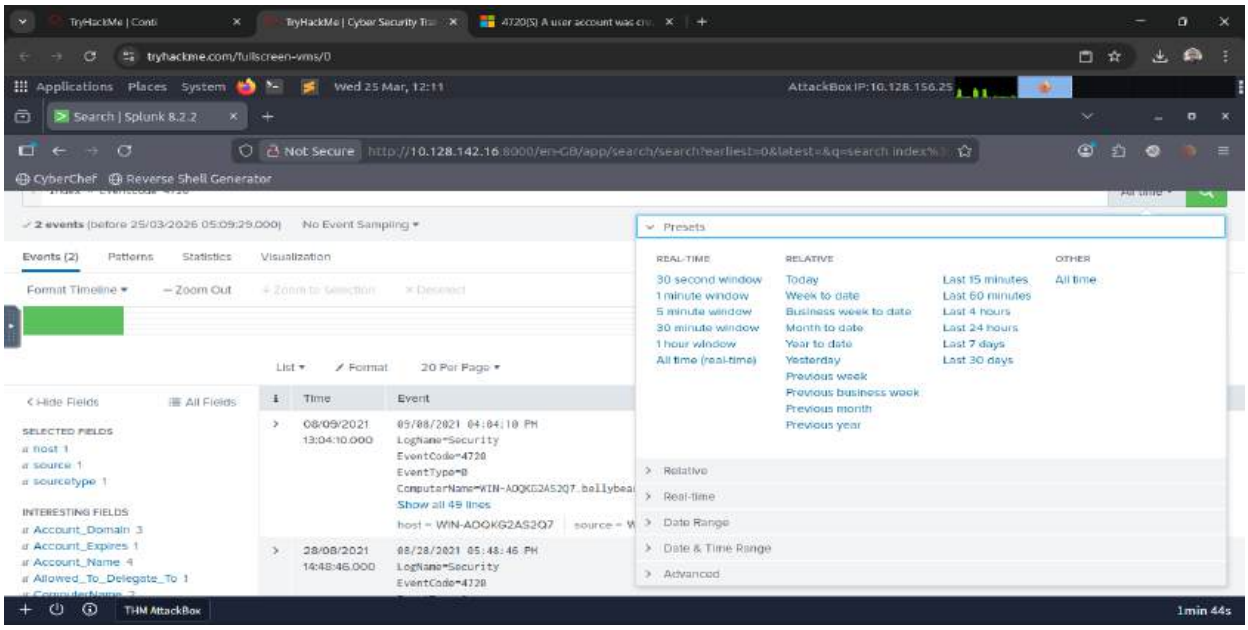

Figure 24: Date & Time Range picker opened, filtering to narrow the two EventCode=4720 events by timestamp

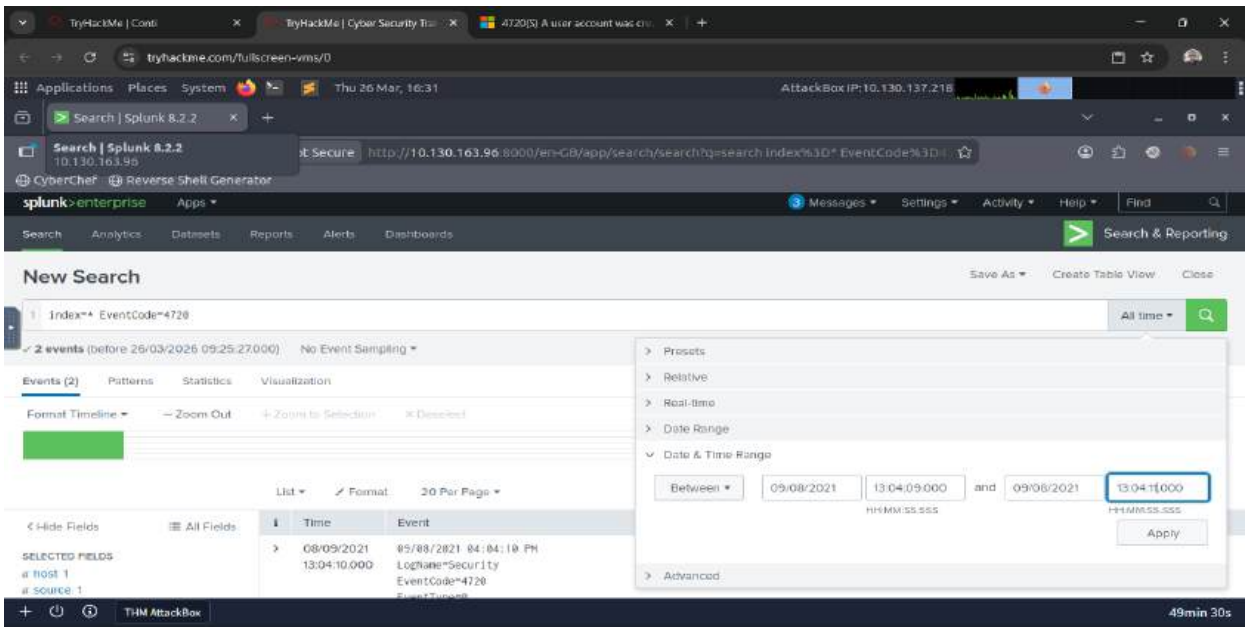

Figure 25: Date & Time Range applied - 09/08/2021 13:04:09 to 13:04:11. Zero results. Range was too narrow — adjusted.

The narrow time window returned nothing. I broadened the approach and therefore switched to EventCode=1 (Process Creation) which captures CommandLine, and searched directly for the exact command string.

```
index=* EventCode=1
# 629 process creation events

index=* EventCode=1 net user /add securityninja hardToHack123$
# Pinpoints the exact command
```

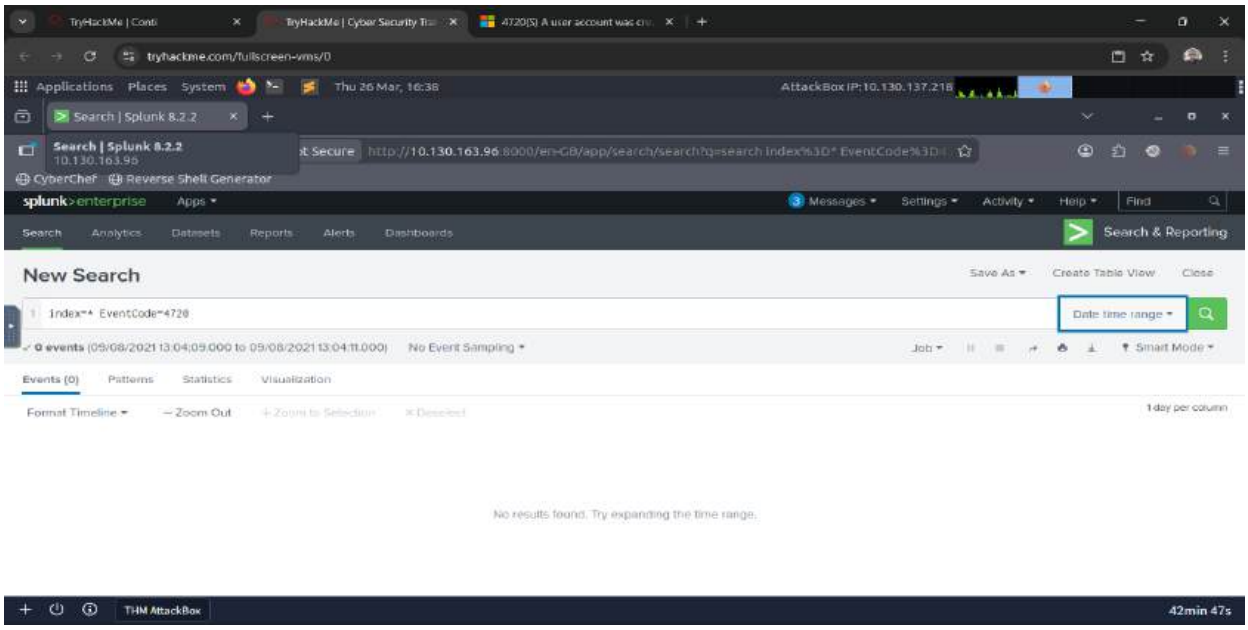

Figure 26: index= EventCode=1 — 629 process creation events, broad scope*

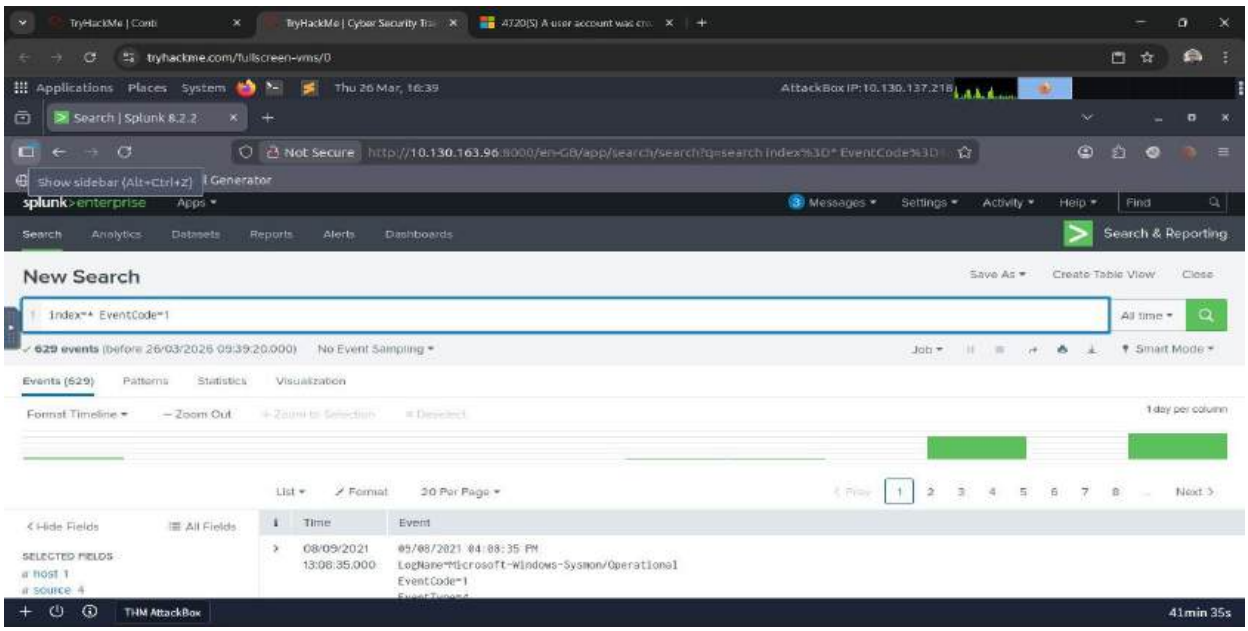

Figure 27: index= EventCode=1 net user /add securityninja hardToHack123$ - 2 events confirming the full command*

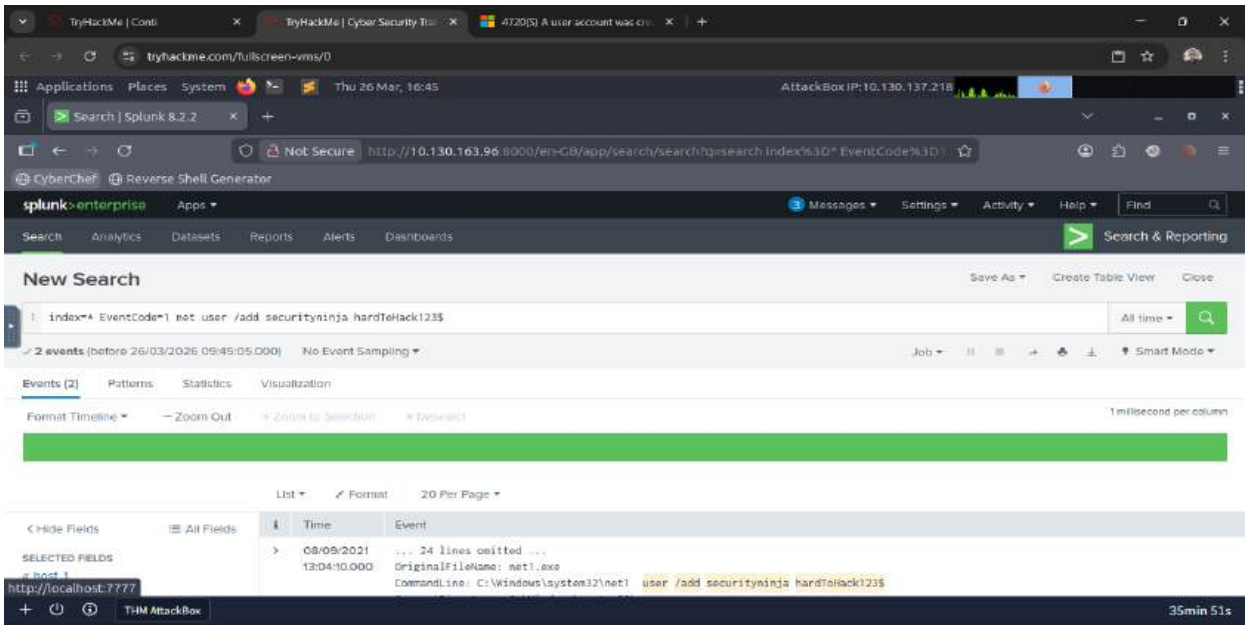

Figure 28: CommandLine field popup - two values: full path version and short net user /add version

Command used to add a new user is `net user /add securityninja hardToHack123$`

SOC Note: The password 'hardToHack123$' is deliberately ironic, attackers sometimes use obvious passwords on backdoor accounts because they don't expect the account to be scrutinised. The real detection signal is the account creation event itself, not the password strength.

▸ Step 7 - Process Migration (CreateRemoteThread - EventCode 8)

Process migration is a post-exploitation technique. After gaining initial access, an attacker migrates their implant into a different process which is usually more stable and trusted process. This makes the malicious code harder to detect because it appears to run under a legitimate process name.

Sysmon Event ID 8 (CreateRemoteThread) captures exactly this - one process injecting a thread into another.

```
index=* EventCode=8
```

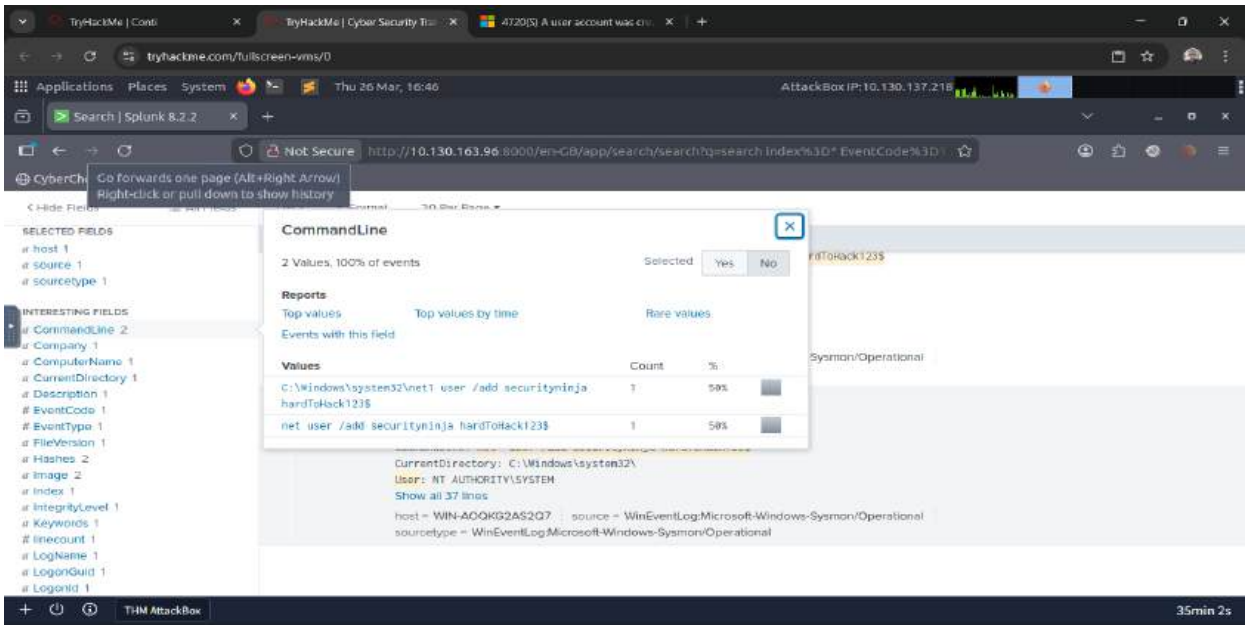

Figure 29: index= EventCode=8 - 2 events returned. Computer: WIN-AOQKG2AS2Q7.bellybear.local*

Two CreateRemoteThread events. Expanding them revealed the source and target process pairs showing the migration chain from PowerShell into unsecapp.exe, and from unsecapp.exe into lsass.exe.

Migrated process image and original process image

`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe, C:\Windows\System32\wbem\unsecapp.exe`

SOC Note: unsecapp.exe is a legitimate WMI (Windows Management Instrumentation) process. Migrating into it gives the attacker a process that is almost never flagged as suspicious, it is expected to be running on most Windows systems. This is a deliberate choice to blend in.

▸ Step 8 - Hash Dumping via LSASS

The same EventCode=8 results revealed the second migration from unsecapp.exe into lsass.exe. I needed to understand why lsass.exe was the target.

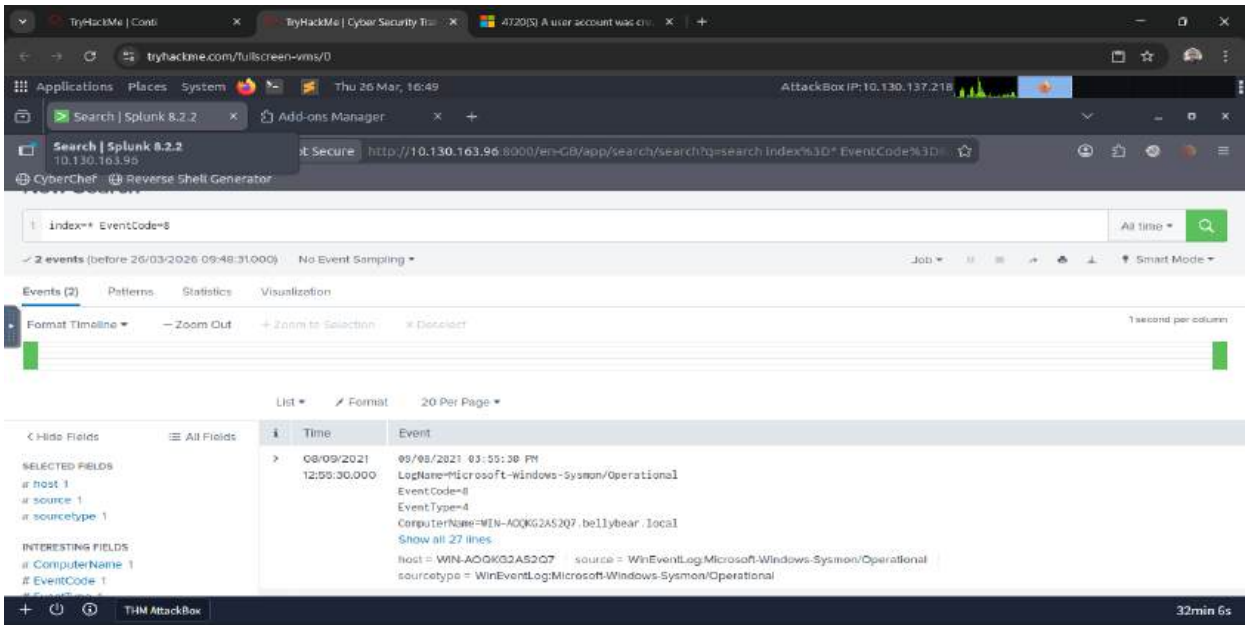

Figure 30: Googling LSASS - Local Security Authority Subsystem Service manages authentication and stores credentials in memory

LSASS (lsass.exe) is the Windows process responsible for authentication. It stores password hashes, Kerberos tickets, and NTLM credentials in memory for active logon sessions. Injecting into lsass.exe gives an attacker direct access to dump all of these, extracting every credential from every currently logged-in user on the machine.

This is one of the most impactful things an attacker can do on a compromised Windows machine. With domain credentials harvested from LSASS, lateral movement across the entire network becomes trivial.

Process image used to get system hashes is `C:\Windows\System32\lsass.exe`

▸ Step 9 - The Web Shell

Exchange servers expose web interfaces in our case being Outlook Web Access (OWA) and the Exchange Admin Center. A web shell is a malicious file (usually ASPX or PHP) uploaded to a web server that gives the attacker remote command execution through a browser. Once a web shell is on an Exchange server, the attacker can run any command as the web server's service account.

I searched for ASPX files in the Sysmon logs.

```
index=* sourcetype="WinEventLog:Microsoft=Windows-Sysmon/Operational" *aspx
```

[Screenshot SPLUNK_31 - screen was black/unavailable during this step]

One event. The filename `i3gfPctK1c2x.aspx` - a randomly generated string designed to be unguessable, dropped into the Exchange OWA auth directory. The auth directory is where Exchange keeps its own authentication pages, making a file there much less likely to be scrutinised during a quick review.

The CommandLine associated with this event used `attrib.exe -r` — removing the read-only attribute from the file, making it writable and executable. This is preparation for the web shell to function.

Web shell filename `i3gfPctK1c2x.aspx`

Command line that executed the web shell

```
attrib.exe -r \\\\win-aoqkg2as2q7.bellybear.local\C$\Program
Files\Microsoft\Exchange
Server\V15\FrontEnd\HttpProxy\owa\auth\i3gfPctK1c2x.aspx
```

SOC Note: The Exchange OWA auth directory (\FrontEnd\HttpProxy\owa\auth\) is a known web shell drop location in ProxyLogon and ProxyShell attacks. Any .aspx file appearing in that directory that is not part of the Exchange installation is a web shell until proven otherwise.

▸ Step 10 - The Three CVEs (External Research)

The challenge required external research to identify the three CVEs. I went to SentinelOne's Conti anthology and Securin.io's analysis to understand what these vulnerabilities actually were and why Conti used them.

[Screenshot SPLUNK_32 - screen was black/unavailable during this step]

CVE-2020-0796 — SMBGhost / CoronaBlue (CVSS 10.0 — Critical)

A remote code execution vulnerability in Windows SMBv3 (Server Message Block version 3). An unauthenticated attacker could send a specially crafted packet to a vulnerable SMB server and execute code with SYSTEM privileges — no credentials required. Also known as SMBGhost and CoronaBlue. It affected Windows 10 and Windows Server 2016/2019. A patch was available from March 2020 but many organisations were slow to apply it. Conti operators used this to move laterally across networks after initial access.

CVE-2018-13379 — Fortinet FortiOS Path Traversal (CVSS 9.8 — Critical)

A critical path traversal vulnerability in Fortinet's FortiOS SSL VPN web portal. An unauthenticated remote attacker could send a specially crafted HTTP request to read system files — including the session file which contains plaintext VPN credentials. This gave Conti operators valid VPN usernames and passwords without needing to brute force anything. Initial access in under a minute. This CVE was actively exploited years after its disclosure because organisations failed to patch their VPN appliances.

CVE-2018-13374 — Fortinet FortiOS Improper Access Control (CVSS 8.8 — High)

Another FortiOS vulnerability allowing an authenticated user to modify the administrative password of another account through an improper access control flaw. Used in combination with CVE-2018-13379 — first steal credentials via path traversal, then use this vulnerability to escalate and gain administrative control of the VPN infrastructure.

CVEs exploited `CVE-2020-0796, CVE-2018-13374, CVE-2018-13379`

The combination of a VPN credential theft (13379) + VPN privilege escalation (13374) + lateral SMB movement (0796) is a complete attack chain: get in through the VPN, escalate on the VPN infrastructure, then spread across the internal network via SMB. No phishing required.

Full Attack Chain - Conti on Exchange

#	PHASE	WHAT HAPPENED

1	Initial Access	CVE-2018-13379 exploited on Fortinet VPN - plaintext credentials stolen from session file. CVE-2018-13374 used to escalate VPN admin access.
2	Lateral Movement	CVE-2020-0796 (SMBGhost) used to move from VPN to internal Exchange server. No credentials needed for this hop.
3	Web Shell Deployment	i3gfPctK1c2x.aspx dropped into Exchange OWA auth directory. attrib.exe -r used to make it executable. Remote command execution established.
4	Persistence	New account 'securityninja' created with password hardToHack123$. Added to Remote Desktop Users AND local Administrators group.
5	Process Migration	PowerShell migrated into unsecapp.exe (WMI process) via CreateRemoteThread. Blends into legitimate Windows activity.
6	Credential Harvesting	unsecapp.exe injected into lsass.exe. All active session credentials from NTLM hashes, Kerberos tickets dumped from memory.
7	Ransomware Deployment	cmd.exe (trojanised) deployed to C:\Users\Administrator\Documents\. Ransomware executed and readme.txt dropped across all user directories.
8	Impact	Exchange server encrypted. OWA and Exchange Admin Center inaccessible. Employees locked out of Outlook.

What I Learned - Real Takeaways

SKILL / CONCEPT	WHAT IT TAUGHT ME
Splunk field exploration	Before querying for specifics, explore Account_Domain, source, sourcetype. It tells you what data is available and saves wasted searches.
Image field for process hunting	The Image field in Sysmon logs is one of the most valuable fields for detecting masquerading. Legitimate processes in wrong locations = instant red flag.
EventCode=11 (FileCreate)	Ransomware drops files. EventCode=11 catches every file creation. Filter by Image to see which process is doing the dropping.
EventCode=4720 + EventCode=1	4720 tells you an account was created. EventCode=1 with CommandLine tells you exactly what command created it. Use both together.
EventCode=8 (CreateRemoteThread)	Process injection leaves a trace. EventCode=8 captures SourceImage and TargetImage - the full injection pair. This is how you find process migration.
LSASS targeting	Any process injecting into lsass.exe is credential dumping. Full stop. No legitimate software does this. It is always an escalation.
Web shell in Exchange OWA auth	The \owa\auth\ directory is a known drop point. Any .aspx file there not in the original Exchange installation is a web shell.
CVE chaining	Conti used three CVEs in sequence - VPN credential theft + VPN privilege escalation + SMB lateral movement. Understanding the chain matters more than knowing each CVE in isolation.

ESCALATION REQUIRED

SOC Incident Report

Conti Ransomware - Exchange Server Compromise

CVE-2020-0796 · CVE-2018-13374 · CVE-2018-13379

Analyst	Mojalefa “Jeff” Letsoara
Date	25 - 26 March 2026
Severity	CRITICAL - Immediate Action Required
Affected System	WIN-AOQKG2AS2Q7.bellybear.local - Exchange Server
Tool Used	Splunk 8.2.2 - 28,145 events analysed
Dwell Time	Minimum 11 days before encryption triggered
Status	ESCALATION REQUIRED

Executive Summary

The Exchange server at WIN-AOQKG2AS2Q7.bellybear.local was compromised by a Conti ransomware operator. The attacker exploited three CVEs to gain access, deployed a web shell on the Exchange OWA interface, created a privileged backdoor account, dumped credentials from LSASS, and encrypted the server. Employees lost access to Outlook and the Exchange Admin Center. The attacker had been present in the environment for at least 11 days before triggering encryption.

11-day dwell time confirmed. Assume data exfiltration occurred before encryption. Treat all domain credentials as compromised.

How They Got In

The attacker chained three CVEs in sequence. CVE-2018-13379 (Fortinet FortiOS, CVSS 9.8) allowed unauthenticated reading of the VPN session file which plaintext credentials extracted with no brute forcing required. CVE-2018-13374 (CVSS 8.8) was used to escalate to VPN administrative access. CVE-2020-0796, SMBGhost (CVSS 10.0), enabled unauthenticated lateral movement from the VPN to the internal Exchange server. No phishing. No user interaction. Three vulnerabilities, one chain.

What They Did Once Inside

The attacker dropped a web shell (i3gfPctK1c2x.aspx) into the Exchange OWA authentication directory, giving persistent browser-accessible command execution. They then created a backdoor local account - securityninja / hardToHack123$ - added it to both the Remote Desktop Users and local Administrators groups, and used PowerShell to migrate their implant into unsecapp.exe before injecting directly into lsass.exe to dump all active session credentials. With domain credentials in hand, they executed the ransomware which is a trojanised cmd.exe placed in C:\Users\Administrator\Documents\ and dropped readme.txt ransom notes across every user directory on the machine.

Key Findings

FINDING	DETAIL
Ransomware location	`C:\Users\Administrator\Documents\cmd.exe`
MD5 hash	`290C7DFB01E50CEA9E19DA81A781AF2C`
Web shell	`\owa\auth\i3gfPctK1c2x.aspx`
Backdoor account	`securityninja - local admin + RDP access`
Credential dumping	`lsass.exe targeted via CreateRemoteThread`
Ransom note	`readme.txt — all user directories`
Dwell time	`11 days minimum before encryption`
CVE — Initial Access	`CVE-2018-13379 (Fortinet FortiOS path traversal)`
CVE — Privilege Escalation	`CVE-2018-13374 (Fortinet FortiOS access control)`
CVE — Lateral Movement	`CVE-2020-0796 (Windows SMBGhost CVSS 10.0)`

LSASS ACCESS CONFIRMED - Every NTLM hash and Kerberos ticket from every active session is in the attacker's hands. Domain-wide password reset is non-negotiable.

Recommendations

Immediate - Do These Now

Patch CVE-2018-13379 and CVE-2018-13374 on the Fortinet FortiOS appliance. These are the entry point. Until patched, the same access vector remains open. Patch CVE-2020-0796 across all internal Windows hosts - any unpatched machine is a lateral movement hop away from the next victim. Rotate every VPN credential in the environment. CVE-2018-13379 exposed plaintext session credentials and it is not known how many accounts were harvested before this incident was detected.

Short Term - Within 72 Hours

Initiate a full domain-wide password reset. LSASS was accessed directly - every NTLM hash and Kerberos ticket from every active session on that server is in the attacker's hands. Audit all local administrator accounts across every host in the environment. The securityninja account was created on this server, there may be others on other machines not yet discovered. Review all ASPX files in Exchange web directories against a known-good baseline. One web shell was found - verify there are no others..

Detection Improvements - Close These Gaps

Implement alerting on Sysmon EventCode 8 targeting lsass.exe. Any process injecting a remote thread into LSASS is credential dumping, this should be an immediate high-severity alert with no exceptions. Implement alerting on ASPX file creation in Exchange web directories. No legitimate Exchange operation writes new ASPX files to the OWA auth folder post-installation. Alert on process image paths outside of expected directories - cmd.exe, powershell.exe, and other system binaries running from user-writable locations should trigger automatically. Monitor EventCode 4720 and follow up on every new local account creation, particularly any account immediately added to the Administrators or Remote Desktop Users group.

Structural - Longer Term

Enforce a vulnerability management cadence with hard SLAs for critical CVEs. CVE-2018-13379 was publicly disclosed in 2018, this server was exploited in 2021. Three years of exposure is not acceptable for a CVSS 9.8 vulnerability on an internet-facing VPN appliance. Implement network segmentation between the VPN infrastructure and internal servers and SMBGhost should not have provided a direct path from VPN to Exchange.

Consider deploying Windows Defender Credential Guard or configuring lsass.exe as a Protected Process Light, which prevents user-mode injection even from administrator-level processes.

PRIORITY	ACTION	RATIONALE
NOW	**Patch FortiOS CVE-2018-13379 & CVE-2018-13374**	Entry point still open until patched
NOW	**Patch CVE-2020-0796 all Windows hosts**	Every unpatched host is a lateral movement target
NOW	**Rotate all VPN credentials**	Plaintext passwords confirmed exposed
NOW	**Isolate Exchange server**	Contain active compromise
72 HRS	**Full domain password reset**	LSASS dumped - all hashes compromised
72 HRS	**Audit all local admin accounts**	securityninja may not be the only backdoor
72 HRS	**Audit all Exchange ASPX files**	Verify no additional web shells
DETECT	**Alert on EventCode 8 >>> lsass.exe**	LSASS injection = credential dumping, always
DETECT	**Alert on ASPX writes in OWA auth dir**	No legitimate Exchange operation does this
DETECT	**Alert on system binaries outside System32**	Masquerading via wrong path - instant red flag
LONG TERM	**Network segmentation VPN >>> internal**	SMBGhost should not reach Exchange directly
LONG TERM	**Credential Guard / PPL for LSASS**	Prevents user-mode LSASS injection entirely
LONG TERM	**CVE SLA enforcement - critical = 30 days**	CVE-2018-13379 was 3 years old. Unacceptable.

Start with the patches and the credential reset.

Everything else builds on those two actions being done first.

Analyst: Mojalefa "Jeff" Letsoara // March 2026 // Status: Escalation Required